



ASCII

Pengenalan C++



A. Struktur Dasar C++

```
helloworld.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  int main () {
5      cout << "Hello, World!" << endl;
6      return 0;
7  }
8
```

Annotations in the image:

- `#include <iostream>` is labeled **include**.
- `using namespace std;` is labeled **namespace**.
- The `int main () { ... }` block is labeled **main**.

- **Include** : Bagian ini berisi daftar *library* yang akan digunakan dalam program. *Library* tersebut dapat berupa *library* bawaan C++ maupun *library* yang dibuat sendiri. Dalam Python, ini serupa dengan *import* untuk memanggil modul-modul yang diperlukan.

- **Namespace** : Penggunaan *namespace* sebenarnya opsional, tetapi sering digunakan dalam pemrograman C++. Pada contoh di atas, kita memakai namespace `std` karena fungsi-fungsi dari *library* `iostream` dan `string` berada di dalam `std`. Tanpa *namespace*, kita harus menuliskan `std::` setiap kali memakai fungsi tersebut.

```
std::cout << "Hello, World!" << std::endl;
```

- **main()** : Bagian ini adalah bagian yang paling penting. Di dalamnya kita tulis semua kode program kita. Program kita akan dijalankan dari sini. Ini wajib ada di setiap program C++.

- **{ dan }** : tempat dimulainya satu blok program.

- **Comment** : Comment adalah bagian dari kode program yang tidak akan dijalankan oleh compiler. Ada dua jenis Comment di C++, yaitu :

- **Single Line Comment** : `//` berlaku untuk satu baris kode



- **Multi Line Comment** : ‘ /* ‘ dan ‘ */ ‘ Comment ini berlaku untuk beberapa baris kode

B. Input dan Output

Output adalah informasi yang udah kita proses dan kita tampilkan ke layar. Di C++, kita bisa menampilkan output ke layar dengan menggunakan ‘**cout**’.

Struktur fungsi ‘**cout**’ :

Data yang ingin ditampilkan

Tuk buat baris baru

```
cout << "Hello, World!" << endl;  
cout << "Belajar C++ sangat seru😊";
```

Semua yang ada di belakang simbol "<<", akan dimasukkan kedalam "cout", kemudian "cout" akan menampilkan data yang dimasukkan

Setelah simbol ‘ << ‘, kita bisa menambahkan data yang mau kita tampilkan. Data ini bisa berupa string, angka, atau variabel.

Contoh:

```
#include <iostream>  
using namespace std;  
int main() {  
    cout << "Hello, World! ";  
    cout << "Belajar C++ sangat seru😊";  
    return 0;  
}
```

Maka, output-nya:



```
Hello, World! Belajar C++ sangat seru😊
```

Kalau mau buat baris baru, kita bisa menggunakan **endl**.

Contoh:

```
cout << "Hello, World!" << endl;  
cout << "Belajar C++ sangat seru😊";
```

Maka outputnya akan nampilin dua baris:

```
Hello, World!  
Belajar C++ sangat seru😊
```

Nahh **simbol** << ini bisa kita sebut sebagai operator insertion atau operator masukan. Karena dia **memasukkan data ke dalam cout**. Kalau di kasus kita di atas, dia menambahkan string ke dalam cout. Kita juga bisa nampilin angka atau variabel.

Contoh:

```
string nama = 'Kei';  
int umur = 20;  
cout << "Halo, nama saya " << nama << " dan umur saya " << umur <<  
"tahun";
```

Maka outputnya akan seperti ini:

```
Halo, nama saya Kei dan umur saya 20 tahun
```



Input adalah data yang kita terima dari user. Di C++, kita bisa menerima input dari user dengan menggunakan 'cin'.

Struktur fungsi 'cin' :



Setelah simbol >>, kita bisa menambahkan variabel yang akan kita isi dengan data yang diinputkan oleh user.

Contoh:

```
#include <iostream>
using namespace std;
int main() {
    string nama;
    cout << "Masukkan nama kamu: ";
    cin >> nama;
    cout << "Halo, " << nama;
    return 0;
}
```



Output:

```
Masukkan nama kamu: Rafli  
Halo, Rafli
```

Namun, fungsi `cin` hanya menerima inputan sampai ketemu spasi. Jadi, kalau kita inputkan Rafli Pernanda, ia hanya menerima Rafli. Kalau kita mau menerima inputan sampai baris baru, kita bisa gunakan ***getline***.

Contoh:

```
string nama;  
cout << "Masukkan nama kamu: ";  
getline(cin, nama);  
cout << "Halo, " << nama;
```

Output:

```
Masukkan nama kamu: Rafli Pernanda  
Halo, Rafli Pernanda
```

Catatan: Perhatikan simbol `<<` itu untuk output dan `>>` itu untuk input.



C. Tipe Data

Tipe data adalah jenis data yang bisa disimpan di dalam variabel. Di C++, ada beberapa tipe data yang bisa kita gunakan. Umumnya pada bahasa pemrograman C++, terdapat tiga jenis tipe data, yaitu :

1. Tipe data primitif

Tipe data primitif adalah tipe data dasar yang digunakan untuk operasi dasar. Contohnya int, boolean, float,

double, char, dan lain-lain. Masing masing tipe data mempunyai karakteristik dan range yang berbeda.

Tipe Data	Ukuran (byte)	Nilai
int	4	9
float	4	9.9
double	8	9.9
char	1	'a'
bool	1	true/false
string	-	"Hello, World!"

Berikut contoh implementasi kode:

```
int umur = 20;
int sumbuY = -45;
float berat_badan = 50.5;
double tinggi_badan = 170.5;
char jenis_kelamin = 'L';
bool is_menikah = false;
string nama = "Aliyyu";
```



Untuk mengetahui ukuran byte setiap tipe data kita dapat menggunakan **operator sizeof** diikuti dengan tipe data yang ingin dicari ukuran byte-nya.

Contoh:

```
#include <iostream>
using namespace std;

int main() {
    cout << "Size of char: " << sizeof(char) << endl;
    cout << "Size of int: " << sizeof(int) << endl;
    cout << "Size of float: " << sizeof(float) << endl;
    cout << "Size of double: " << sizeof(double) << endl;
    cout << "Size of long: " << sizeof(long);
}
```

Output:

```
Size of char: 1
Size of int: 4
Size of float: 4
Size of double: 8
Size of long: 8
```

2. Tipe data kolektif

Tipe data kolektif adalah tipe data yang berupa rangkaian atau kumpulan data yang memiliki indeks, seperti

array atau list.



```
int angka[5] = {1, 2, 3, 4, 5};
```

3. Tipe data abstrak

Contoh dari tipe data abstrak adalah struct. Struct adalah sekumpulan variabel bertipe data abstrak yang dinyatakan dengan sebuah nama dan bisa diciptakan secara dinamis.

```
struct Mahasiswa {  
    string nama;  
    int umur;  
    float ipk;  
};  
Mahasiswa mhs = {"Sipuwah", 20, 3.5};
```

D. VARIABEL

Segala program komputer yang sedang berjalan akan menyimpan data sementara di dalam RAM (Random Access Memori). Data-data yang tersimpan dalam RAM punya alamat yang direpresentasikan dalam bentuk hexadecimal. Nah, biar gampang, kita kasih nama ke alamat-alamat ini. Nama ini kita sebut sebagai variabel.

1. Cara Buat Variabel

Untuk membuat variabel, kita harus menentukan tipe datanya dulu dan kasih nama variabelnya. Struktur penulisan variabel di C++:

```
//tipe_data nama_variabel;  
int umur;  
// atau  
//tipe_data nama_variabel = nilai;  
string nama = "Hannah";
```



Pada variabel pertama yaitu umur memiliki nilai awal tetapi isinya null (kosong), sedangkan variabel kedua yaitu nama memiliki nilai awal yaitu Hannah.

2. Aturan penulisan variabel

Aturan	Contoh Benar	Contoh Salah	Penjelasan
Harus diawali huruf atau underscore	umur, _data	1umur, \$gaji	Tidak boleh mulai dengan angka atau simbol selain _
Tidak boleh dimulai angka	nilai1	2nilai	Angka hanya boleh setelah huruf/underscore
Tidak boleh ada spasi	totalHarga	Total harga	Spasi akan dianggap pemisah perintah
Tidak boleh karakter spesial selain underscore	berat_badan	Berat badan	Dua kata harus disambungkan
Case sensitive	Umur, umur, UMUR (3 variabel berbeda)	—	C++ membedakan huruf besar dan kecil
Tidak boleh pakai keyword C++	data, input_user	for, if, while	Keyword sudah digunakan oleh bahasa C++
Boleh menggunakan underscore di awal/akhir	_nama, nama_akhir	nama akhir	Underscore aman dipakai

E. Operator

1. Operator Aritmatika



Operator aritmatika digunakan untuk melakukan operasi matematika.

Penjumlahan	+
Pengurangan	-
Perkalian	*
Pembagian	/
Sisa bagi / Modulus	%
Increment	++
Decrement	--

Contoh:

```
#include <iostream>
using namespace std;
int main() {
    int a, b, hasil;
    cout << "Masukkan nilai a: ";
    cin >> a;
    cout << "Masukkan nilai b: ";
    cin >> b;
    // Penjumlahan
    hasil = a + b;
    cout << "Hasil penjumlahan: " << hasil << endl;
    // Pengurangan
    hasil = a - b;
    cout << "Hasil pengurangan: " << hasil << endl;
    // Perkalian
    hasil = a * b;
    cout << "Hasil perkalian: " << hasil << endl;
    // Pembagian
    hasil = a / b;
```



```
cout << "Hasil pembagian: " << hasil << endl;
// Modulus
hasil = a % b;
cout << "Hasil modulus: " << hasil << endl;
return 0;
}
```

Output:

```
Masukkan nilai a: 6
Masukkan nilai b: 7
Hasil penjumlahan: 13
Hasil pengurangan: -1
Hasil perkalian: 42
Hasil pembagian: 0
Hasil modulus: 6
```

2. Operator Penugasan

Operator Penugasan atau disebut juga Assignment Operator berfungsi untuk memberikan tugas pada variabel atau untuk mengisi nilai pada variabel tersebut.

Penugasan	=
Penambahan	+=
Pengurangan	-=
Perkalian	*=
Pembagian	/=
Sisa bagi	%=
Pergeseran bit ke kiri	>>=
Pergeseran bit ke kanan	<<=



Bitwise AND	&=
Bitwise OR Eksklusif	^=
Bitwise OR Inklusif	! =

Contoh:

```
#include <iostream>
using namespace std;
int main() {
    int a, b;
    cout << "Masukkan nilai a: ";
    cin >> a;
    cout << "Masukkan nilai b: ";
    cin >> b;
    // Penjumlahan
    a += b;
    cout << "Hasil penjumlahan: " << a << endl;
    // Pengurangan
    a -= b;
    cout << "Hasil pengurangan: " << a << endl;
    // Perkalian
    a *= b;
    cout << "Hasil perkalian: " << a << endl;
    // Pembagian
    a /= b;
    cout << "Hasil pembagian: " << a << endl;
    // Modulus
    a %= b;
    cout << "Hasil modulus: " << a << endl;
    return 0;
}
```

Output:

```
Masukkan nilai a: 12
```



```
Masukkan nilai b: 4
Hasil penjumlahan: 16
Hasil pengurangan: 12
Hasil perkalian: 48
Hasil pembagian: 12
Hasil modulus: 0
```

Catatan: variabel a nilainya terus berganti (tertimpa) hasil perhitungan sebelumnya.

3. Operator Perbandingan

Operator yang digunakan untuk membandingkan dua *operand*, kemudian menghasilkan nilai *boolean*, benar (1) dan juga nilai salah (0). Operator ini sering digunakan dalam struktur kontrol seperti if, else, dan switch.

Sama dengan	==
Tidak sama dengan	!=
Kurang dari	<
Lebih besar dari	>
Kurang dari sama dengan	<=
Lebih dari sama dengan	>=

Contoh:

```
#include <iostream>
using namespace std;

int main() {
    int apel, melon;

    cout << "=== Program Perbandingan Buah ===" << endl;
    cout << "Masukkan jumlah Apel: ";
    cin >> apel;
```



```
cout << "Masukkan jumlah Melon: ";
cin >> melon;

cout << "\n--- Hasil Operator Perbandingan ---" << endl;
cout << boolalpha;

cout << "Apakah jumlah Apel == Melon? : " << (apel == melon) << endl;
cout << "Apakah jumlah Apel != Melon? : " << (apel != melon) << endl;
cout << "Apakah jumlah Apel > Melon? : " << (apel > melon) << endl;
cout << "Apakah jumlah Apel < Melon? : " << (apel < melon) << endl;
cout << "Apakah jumlah Apel >= Melon? : " << (apel >= melon) << endl;
cout << "Apakah jumlah Apel <= Melon? : " << (apel <= melon) << endl;

cout << "\n--- Keputusan Belanja ---" << endl;
if (apel > melon) {
    cout << "Stok Apel lebih banyak, kita jual Apel aja." << endl;
} else if (melon > apel) {
    cout << "Stok Melon lebih banyak, kita jual Melon aja." << endl;
} else {
    cout << "Stok sama banyak, kita jual aja semua" << endl;
}
return 0;
}
```

Output:

```
=== Program Perbandingan Buah ===
Masukkan jumlah Apel: 12
Masukkan jumlah Melon: 15

--- Hasil Operator Perbandingan ---
Apakah jumlah Apel == Melon? : false
Apakah jumlah Apel != Melon? : true
Apakah jumlah Apel > Melon? : false
Apakah jumlah Apel < Melon? : true
Apakah jumlah Apel >= Melon? : false
```



```
Apakah jumlah Apel ≤ Melon? : true
```

```
--- Keputusan Belanja ---
```

```
Stok Melon lebih banyak, kita jual Melon saja.
```

4. Operator Logika

Operator logika digunakan untuk menggabungkan dua atau lebih ekspresi logika. Operator ini menghasilkan nilai true atau false. Operator ini sering digunakan dalam struktur kontrol seperti if, else, dan switch.

AND	&&
OR	
NOT	!

Contoh:

```
#include <iostream>
using namespace std;

int main() {
    int a, b;

    cout << "Masukkan nilai a: "; cin >> a;
    cout << "Masukkan nilai b: "; cin >> b;

    // Dan
    cout << "Apakah a lebih besar dari 0 dan b lebih kecil dari 10? " << (a > 0
    && b < 10) << endl;

    // Atau
    cout << "Apakah a lebih besar dari 0 atau b lebih kecil dari 10? " << (a > 0
    || b < 10) << endl;
```




```
// Tidak
cout << "Apakah a lebih besar dari 0? " << !a << endl;

return 0;
}
```

Output:

```
Masukkan nilai a: 5
Masukkan nilai b: 2
Apakah a lebih besar dari 0 dan b lebih kecil dari 10? 1
Apakah a lebih besar dari 0 atau b lebih kecil dari 10? 1
Apakah a lebih besar dari 0? 0
```

5. Operator Bitwise

Operator bitwise adalah operasi matematika yang mengoperasikan pada bilangan biner berbasis 2.

AND	&
OR	
XOR	^
NOT	~
Pergeseran bit ke kiri	<<
Pergeseran bit ke kanan	>>

6. Operator Ternary



Operator ternary adalah operator yang memungkinkan kita untuk menulis kode if-else dalam satu baris. Operator ini terdiri dari tiga bagian, yaitu kondisi, ekspresi jika benar, dan ekspresi jika salah.

Struktur operator ternary:

```
kondisi ? ekspresi_jika_benar : ekspresi_jika_salah
```

Contoh:

```
#include <iostream>
using namespace std;

int main() {
    bool hujan = false;
    string pesan = hujan ? "Bawa payung" : "Tidak perlu payung";
    cout << pesan;
}
```

Output:

```
Tidak perlu payung
```

6. Operator Increment dan Decrement

Operator increment dan decrement digunakan untuk menambah atau mengurangi nilai variabel sebanyak satu.

Nama Operator	Operator	Bentuk Lain	Contoh
Increment	++	a++	a++ atau ++a
Decrement	--	a--	a-- atau --a



Perhatikan increment `a++` dan `++a` berbeda. `a++` akan menampilkan nilai `a` terlebih dahulu, baru nilai `a` ditambah satu. Sedangkan `++a` akan menambahkan nilai `a` terlebih dahulu, baru nilai `a` ditampilkan. Begitu juga untuk decrement.

Contoh:

```
#include <iostream>
using namespace std;

int main() {
    int z = 1;
    cout << "Sebelum increment " << z << endl;
    cout << "Increment di depan " << ++z << ", sudah bertambah" <<
endl;
    cout << "Increment di belakang " << z++ << ", belum bertambah" <<
endl;
    cout << "Hasil akhir " << z << endl;
    return 0;
}
```

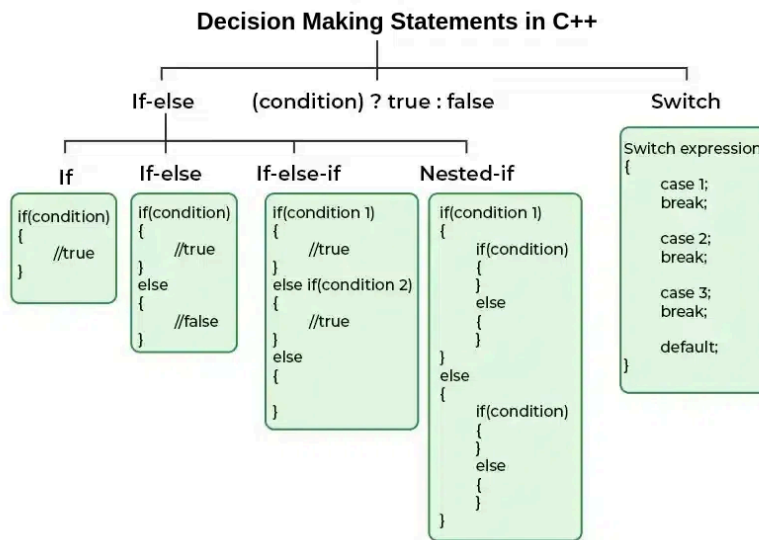
Output:

```
Sebelum increment 5
Increment di depan 6, sudah bertambah
Increment di belakang 6, belum bertambah
Hasil akhir 7
```

F. Percabangan



Percabangan adalah sebuah struktur kontrol yang memungkinkan kita untuk memeriksa kondisi tertentu dan menjalankan kode tertentu berdasarkan apakah kondisi tersebut benar atau salah.



Sumber: [geeksforgeeks.org](https://www.geeksforgeeks.org/)

1. Percabangan if

Percabangan if adalah percabangan yang cuman punya satu blok pilihan ketika kondisinya terpenuhi atau bernilai benar. Struktur penulisan percabangan if di C++:

```
If (kondisi) {
    // kode yang akan dijalankan jika kondisi bernilai benar
}
```

Contoh:

```
#include <iostream>
using namespace std;
int main() {
```



```
bool mengantuk = true;
if (mengantuk) {
    cout << "Saya tidur";
}
return 0;
}
```

Output:

```
Saya tidur
```

2. Percabangan If-else

Percabangan if else adalah percabangan yang punya dua blok pilihan. Satu blok pilihan ketika kondisinya terpenuhi atau bernilai benar, dan satu blok pilihan ketika kondisinya tidak terpenuhi atau bernilai salah. Struktur penulisan percabangan if-else di C++:

```
if (kondisi) {
    // kode yang akan dijalankan jika kondisi bernilai benar
} else {
    // kode yang akan dijalankan jika kondisi bernilai salah
}
```

Contoh:

```
#include <iostream>
using namespace std;

int main() {
    int age = 19;
    if (age > 18)
        cout << "Dewasa";
}
```



```
    return 0;  
}
```

Output:

```
Dewasa
```

3. Percabangan Else-If

Percabangan else if adalah percabangan yang punya lebih dari dua blok pilihan. Kita bisa menambahkan blok pilihan baru ketika kondisi sebelumnya tidak terpenuhi.

```
if (kondisi1) {  
    // kode yang akan dijalankan jika kondisi1 bernilai benar  
} else if (kondisi2) {  
    // kode yang akan dijalankan jika kondisi2 bernilai benar  
} else {  
    // kode yang akan dijalankan jika semua kondisi salah  
}
```

Contoh:

```
#include <iostream>  
using namespace std;  
  
int main() {  
    int age = 18;  
  
    if (age < 13) {  
        cout << "Anak";  
    }  
  
    else if (age ≥ 1 and age ≤ 18) {  
        cout << "Remaja";  
    }  
}
```



```
else {  
    cout << "Dewasa";  
}  
return 0;  
}
```

Output:

```
Dewasa
```

4. Percabangan switch/case

Percabangan switch/case adalah bentuk lain dari percabangan if/else/if. Untuk bagian case, kita bisa buat sebanyak mungkin sesuai dengan kebutuhan. Jika kita tidak menemukan nilai yang sesuai, default akan digunakan. Setiap case harus diakhiri dengan break. Tujuannya agar berhenti mengecek case selanjutnya, ketika case saat ini sudah terpenuhi. Untuk default tak perlu diakhiri break.

Berikut struktur penulisan switch case pada C++:

```
switch(variabel) {  
    case <value>:  
        // blok kode  
        break;  
    case <value>:  
        // blok kode  
        break;  
    default:  
        // blok kode  
}
```

Contoh:

```
#include <iostream>
```



```
using namespace std;

int main() {
    int nilai;
    cout << "≡≡≡ Konversi Nilai Akademik ≡≡≡\n";
    cout << "Masukkan nilai akhir (0-100): ";
    cin >> nilai;
    if (nilai < 0 || nilai > 100) {
        cout << "Error: Nilai harus 0-100.\n";
        return 1;
    }

    switch (nilai / 10) {
        case 10:
            cout << "Predikat: A (Sangat Baik)\n";
            break;
        case 9:
            cout << "Predikat: A (Sangat Baik)\n";
            break;
        case 8:
            cout << "Predikat: B (Baik)\n";
            break;
        case 7:
            cout << "Predikat: C (Cukup)\n";
            break;
        case 6:
            cout << "Predikat: D (Kurang)\n";
            break;
        default:
            cout << "Predikat: E (Gagal)\n";
            break;
    }
    return 0;
}
```




5. Nested If

Nested if adalah percabangan if yang ada di dalam if.

Contoh :

```
#include <iostream>
using namespace std;

int main() {
    bool malas = true;
    bool mengantuk = true;
    if (malas) {
        if (mengantuk) {
            cout << "Saya tidur";
        }
    } else {
        cout << "Gak ngapa-ngapain";
    }
}
```

Output:

```
Saya tidur
```

6. Ternary Operator

Ternary operator ini membuat if-else dalam satu baris.

Struktur penulisan ternary operator:

```
(kondisi) ? nilaiJikaBenar : nilaiJikaSalah;
```

Contoh:

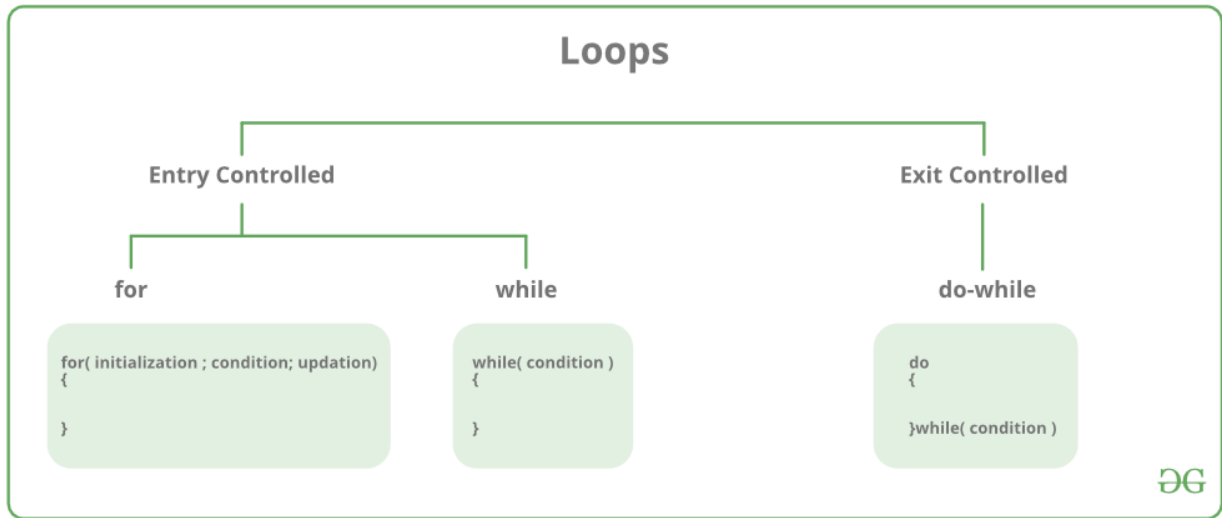
```
bool mengantuk = true;
string aktivitas = (mengantuk) ? "Tidur" : "Belajar";
cout << aktivitas;
```

Output

```
Tidur
```



G. Perulangan



Sumber: [geeksforgeeks.org](https://www.geeksforgeeks.org/)

1. Entry Controlled Loop

kondisi diperiksa di awal sebelum masuk ke dalam badan perulangan. Program mengevaluasi kondisi terlebih dahulu. Jika benar (true), kode di dalamnya dijalankan. Jika salah (false) sejak awal, isi loop tidak akan pernah dijalankan sama sekali.

a. For Loop

Perulangan for merupakan perulangan yang sudah ditentukan dan jelas berapa kali ia akan mengulang.

Struktur dari for loop adalah sebagai berikut:

```
for (initialization; condition; update) {
    // kode yang akan diulang
}
```

- **Initialization** adalah bagian yang akan dieksekusi pertama kali ketika loop dimulai. Biasanya digunakan untuk mendeklarasikan variabel yang akan digunakan dalam loop.
- **Condition** adalah bagian yang akan dievaluasi sebelum setiap iterasi loop. Jika hasil evaluasi ini adalah true, maka loop lanjut. Jika false, maka loop selesai.
- **Update** adalah bagian yang akan dieksekusi setelah setiap iterasi loop.



Biasanya kita gunakan untuk update variabel yang digunakan dalam loop.

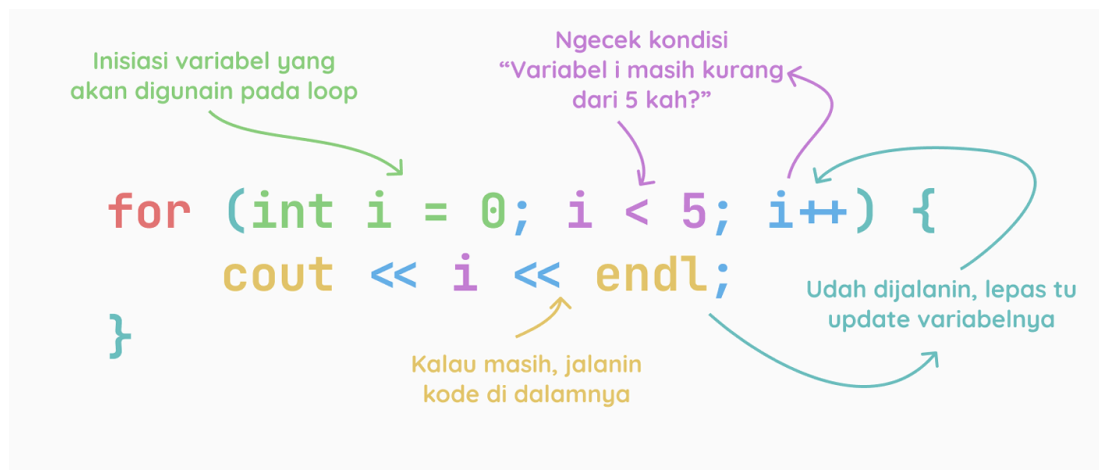
Contoh:

```
#include <iostream>
using namespace std;
int main() {
    for (int i = 0; i < 5; i++) {
        cout << i << endl;
    }
    return 0;
}
```

Output:

```
0
1
2
3
4
```

Ilustrasi:



b. While Loop



While merupakan perulangan uncounted loop atau tidak ditentukan berapa kali mengulangnya. Perulangan ini akan terus dilakukan selama kondisinya terpenuhi. Struktur dari for loop adalah sebagai berikut:

```
while (condition) {  
    // kode yang akan diulang  
}
```

Condition adalah bagian yang akan dievaluasi sebelum setiap iterasi loop. Jika hasil evaluasi ini adalah true, maka loop lanjut. Jika false, maka loop berhenti.

Contoh:

```
#include <iostream>  
using namespace std;  
  
int main() {  
    int i = 0;  
    while (i < 5) {  
        cout << i << endl; i++; }  
    return 0;  
}
```

Output:

```
0  
1  
2  
3  
4
```



Ilustrasi:

```
int i = 0;
while (i < 5) {
    cout << i << endl;
    i++;
}
```

Annotations:

- Inisiasi variabel yang akan digunakan pada loop (points to `int i = 0;`)
- Ngecek kondisi "Variabel i masih kurang dari 5 kah?" (points to `i < 5`)
- Kondisi tidak selalu tentang angka! (points to `i < 5`)
- Kalau masih, jalanin kode di dalamnya (points to the loop body)

c. Perulangan Foreach

Foreach-loop ini adalah loop yang digunakan untuk mengakses elemen-elemen dari array atau collection. Struktur dari Foreach loop adalah sebagai berikut:

```
for (tipe_data variabel : array) {
    // kode yang akan diulang
}
```

Contoh :

```
#include <iostream>
using namespace std;
int main() {
    int arr[] = {1, 2, 3, 4, 5};
    for (int x : arr) {
        cout << x << endl;
    }
    return 0;
}
```



Output:

```
1  
2  
3  
4  
5
```

2. Exit Controlled Loop

Exit-controlled loop adalah perulangan yang mengecek kondisi di akhir. Sederhananya, jalankan dulu kodenya sekali, baru tanya kemudian apakah harus lanjut atau berhenti.

A. Do-While

Mirip seperti while, perbedaannya terdapat pada perintah do di awal yang berguna untuk mengeksekusi perintah satu kali terlebih dahulu, kemudian memeriksa kondisi perulangan, apabila benar perintah akan dieksekusi lagi dan jika salah maka perulangan akan berhenti. Struktur dari do-while loop adalah:

```
do {  
    // kode yang akan diulang  
} while (condition);
```

Condition adalah bagian yang akan dievaluasi setelah setiap iterasi loop. Jika hasil evaluasi ini adalah true, maka loop lanjut. Jika false, maka loop udahan.

Contoh:

```
#include <iostream>  
using namespace std;  
  
int main() {
```

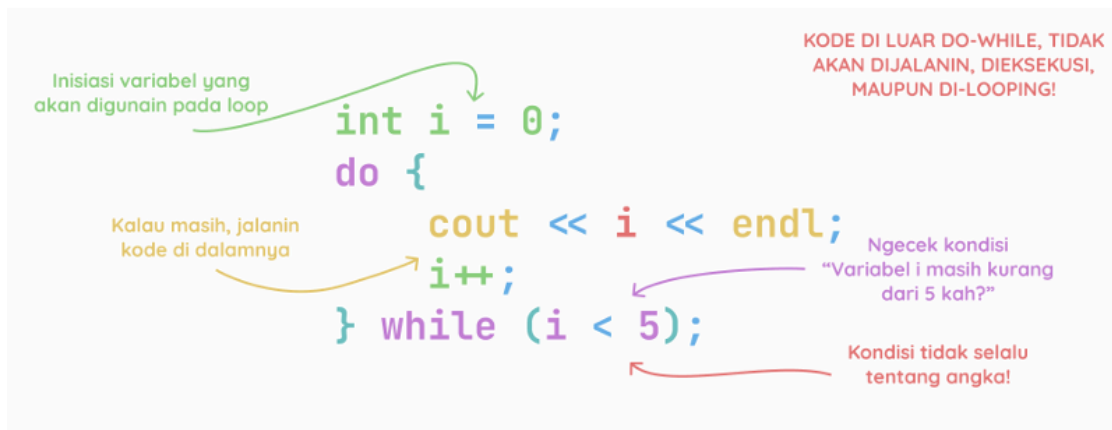


```
int i = 0;
do {
    cout << i << endl;
    i++;
} while (i < 5);
return 0;
}
```

Output:

```
0
1
2
3
4
```

Ilustrasi:





H. Flowchart

Flowchart (Bagan Alur) adalah diagram yang menampilkan langkah-langkah, instruksi, dan keputusan dalam suatu proses atau program. Setiap tahapan digambarkan dalam bentuk simbol visual yang dihubungkan dengan garis atau panah untuk menunjukkan arah aliran kerja.

1. Fungsi Utama

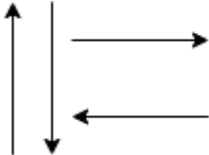
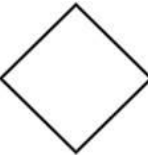
- **Visualisasi Program:** Memberi gambaran jalannya program dari satu proses ke proses lainnya.
- **Kemudahan Pemahaman:** Membuat alur logika yang kompleks menjadi lebih mudah dipahami oleh semua orang.
- **Penyederhanaan Prosedur:** Merangkum rangkaian prosedur yang panjang agar informasi lebih cepat terserap.

2. Panduan Membuat Flowchar

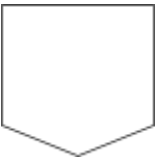
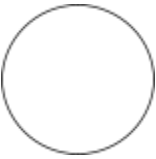
Agar flowchart efektif dan standar, perhatikan poin-poin berikut:

- **Arah Aliran:** Dibuat secara berurutan dari atas ke bawah dan dimulai dari bagian kiri halaman.
- **Kejelasan Kegiatan:** Setiap tahapan atau instruksi harus ditunjukkan dengan jelas dan tidak ambigu.
- **Titik Awal & Akhir:** Harus memiliki titik mulai (Start) dan titik henti (End) yang tegas.
- **Penggunaan Kata Kerja:** Gunakan kata yang mewakili suatu aktivitas atau pekerjaan (misal: "Hitung Luas", "Input Data").
- **Urutan Logis:** Kegiatan harus disusun secara kronologis sesuai dengan urutan kejadiannya.
- **Simbol Penghubung:** Jika flowchart terpotong atau berpindah halaman, gunakan simbol penghubung (connector) agar alur tetap menyambung.

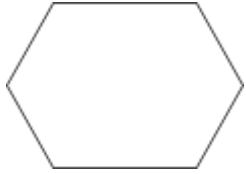


No	Simbol	Nama	Fungsi
1		Flowline	Menunjukkan arah proses serta menggabungkan dua blok
2		Terminal/ terminator	Menyatakan awal atau akhir suatu program
3		Proses	Menyatakan suatu proses yang akan dikerjakan program
4		Data/ input & output	Menyatakan proses input atau output tanpa tergantung peralatan
5		Decision	Menunjukkan kondisi tertentu yang akan bernilai TRUE or False.



6		Database	Menunjukkan tempat penyimpanan data dalam sistem
7		Off-page reference	Menghubungkan dua halaman atau lembar flowchart yang berbeda
8		On-page reference	Merepresentasikan keterkaitan antar dua bagian dalam flowchart yang terpisah namun masih pada lembar flowchart yang sama
9		Predefined process	Menunjukkan proses yang telah ditentukan sebelumnya dan sering digunakan dalam proses yang sama (prosedur)
10		Document	Menunjukkan dokumen atau data tertentu dalam suatu proses



11		Preparation	Menunjukkan persiapan atau pemrosesan awal sebelum masuk ke proses utama dalam suatu sistem atau proses.
12		Alternate process	Menunjukkan proses atau langkah alternatif dalam suatu sistem atau proses jika terjadi kondisi tertentu
13		Manual operation	Menunjukkan tindakan yang dilakukan secara manual oleh manusia
14		Delay	Menggambarkan penundaan atau waktu tunggu dalam suatu sistem atau proses



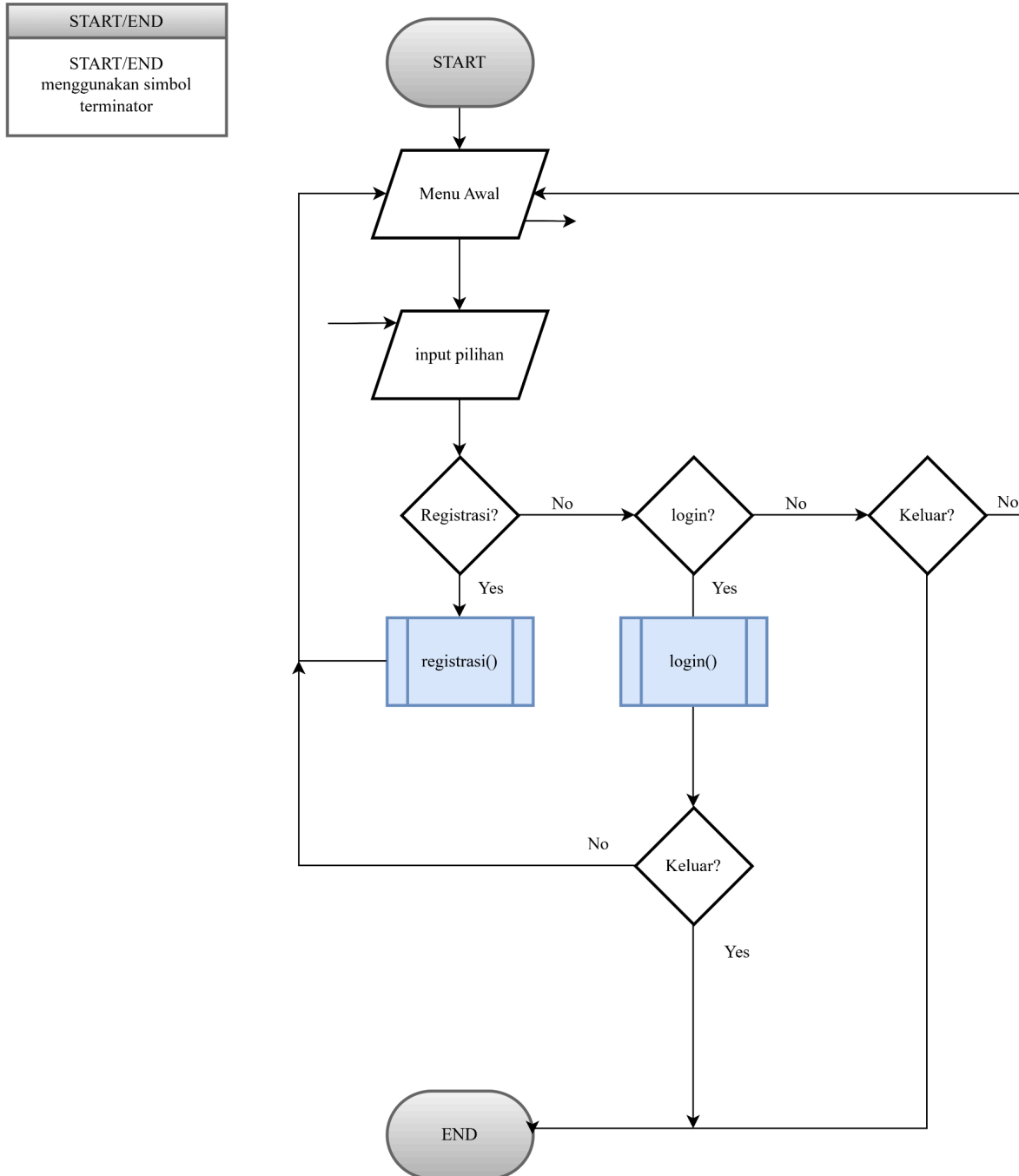
Studi Kasus

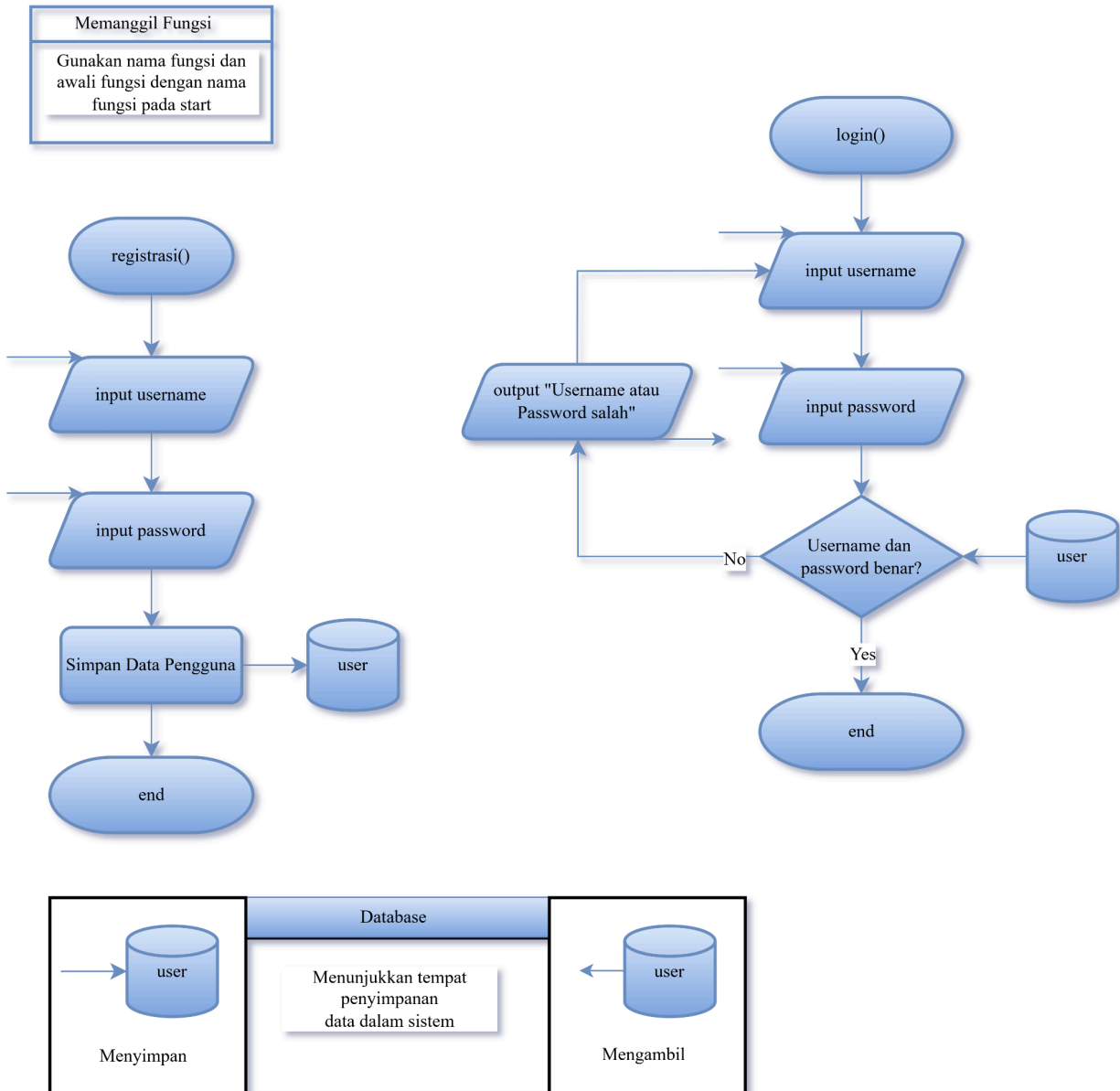
1. Buatlah flowchart, pseudocode, dan program yang meminta input Tahun (angka) dari pengguna. Program harus menentukan apakah tahun tersebut termasuk Tahun Kabisat atau Bukan Tahun Kabisat dengan ketentuan:
 - Jika tahun habis dibagi 400, maka Kabisat.
 - Jika tidak habis dibagi 400, tetapi habis dibagi 100, maka Bukan Kabisat.
 - Jika tidak habis dibagi 100, tetapi habis dibagi 4, maka Kabisat.
 - Selain itu, maka Bukan Kabisat.
2. Buatlah flowchart, pseudocode, dan program kasir sederhana yang memungkinkan pengguna memasukkan Harga Barang secara berulang-ulang. Berikut ketentuannya:
 - Program akan terus meminta input harga hingga pengguna memasukkan angka 0 (sebagai tanda selesai).
 - Program harus menghitung Total Belanja.
 - Kondisi Khusus: Jika Total Belanja lebih dari Rp 100.000, pengguna mendapatkan Diskon 10%. Jika tidak, tidak ada diskon.
 - Tampilkan Total Akhir yang harus dibayar.



Suplemen

Contoh penerapan simbol flowchart.

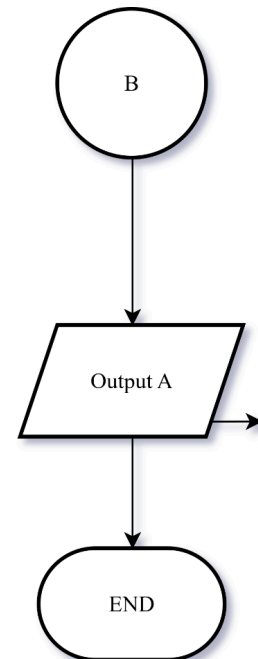
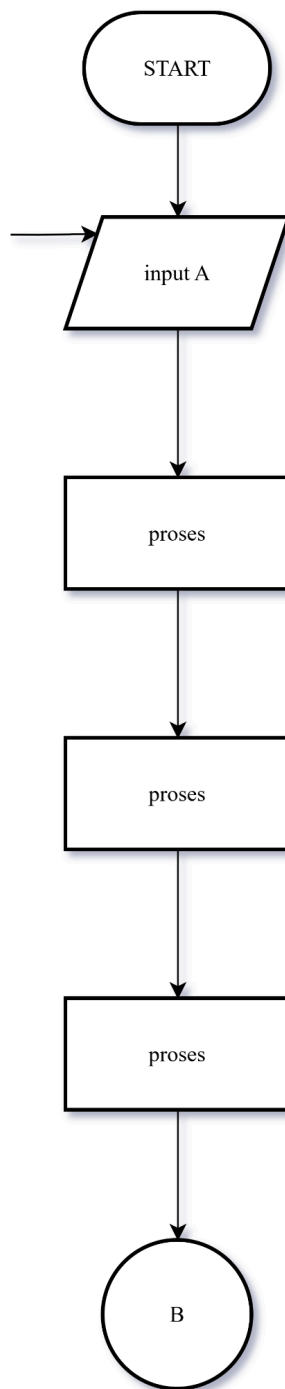






On Page

Apabila flowchart terlalu panjang, gunakan on page jika lanjutannya di dalam halaman yang sama





Off Page

Apabila flowchart terlalu panjang, gunakan off page jika lanjutannya di dalam halaman/lembaran yang berbeda

